

Translated

From: e-maxx.ru

Suffix Array

Definition

Let s be a string of length n . The i -th suffix of s is the substring $s[i \dots n - 1]$.

A **suffix array** will contain integers that represent the **starting indexes** of the all the suffixes of a given string, after the aforementioned suffixes are sorted.

As an example look at the string $s = \text{abaab}$. All suffixes are as follows

$\begin{array}{l} 0. \text{ \& abaab} \\ 1. \text{ \& baab} \\ 2. \text{ \& aab} \\ 3. \text{ \& ab} \\ 4. \text{ \& b} \end{array}$

After sorting these strings:

$\begin{array}{l} 2. \text{ \& aab} \\ 3. \text{ \& ab} \\ 0. \text{ \& abaab} \\ 4. \text{ \& b} \\ 1. \text{ \& baab} \end{array}$

Therefore the suffix array for s will be $(2, \sim 3, \sim 0, \sim 4, \sim 1)$.

As a data structure it is widely used in areas such as data compression, bioinformatics and, in general, in any area that deals with strings and string matching problems.

Construction

$O(n^2 \log n)$ approach

This is the most naive approach. Get all the suffixes and sort them using quicksort or mergesort and simultaneously retain their original indices. Sorting uses $O(n \log n)$ comparisons, and since comparing two strings will additionally take $O(n)$ time, we get the final complexity of $O(n^2 \log n)$.

$O(n \log n)$ approach

Strictly speaking the following algorithm will not sort the suffixes, but rather the cyclic shifts of a string. However we can very easily derive an algorithm for sorting suffixes from it: it is enough to append an arbitrary character to the end of the string which is smaller than any character from the string. It is common to use the symbol $\$$. Then the order of the sorted cyclic